

1 chapter9: 分支限界法

1.1 分支-限界法基本原理

- **分支-限界法定义**: 生成当前 E-结点 (扩展结点) 的全部子结点 (分支), 利用剪枝函数对不能生成含答案结点的子树剪枝 (限界) 的状态空间树检索方法。即, 每当访问一个结点 (E-结点) 时, 会生成这个结点的所有子结点 (广度优先搜索)
- **核心要素**: 多个活节点的组织, E-节点的检索, 剪枝函数的设计
- **FIFO 检索策略**: 类似于 BFS 的状态空间检索, 活结点采用普通队列组织 (first in first out), 选择方式太过死板, 不利于答案的得到。
- **LIFO 检索策略**: 类似于 D-检索的状态空间检索, 活结点表采用栈组织 (last in first out)
- **最优检索策略**: 最小耗费 (最大收益) 的方式检索, 活结点表采用优先级队列组织

目标状态是否可由初始状态到达的判定方法: 对于 15-迷问题 (4*4 棋盘内通过合法移动恢复成目标排列)

给 4*4 棋盘的方格编号 1 ~ 16 位置 i 表示目标排列中放 i 号牌的方格位置, 位置 16 是空格的位置。

$POSITION(i)$: 编号 i 的牌在初始状态的位置号 ($1 \leq i < 16$) $POSITION(16)$: 空格的位置号, $LESS(i)$: 满足 $\{j | j < i \wedge POSITION(j) > POSITION(i)\}$ 的数目。

若初始状态空格横纵坐标和为奇数, $X = 1$; 否则 $X = 0$ 。

当且仅当 $\sum_{i=1}^{16} LESS(i) + X$ 为偶数时, 目标状态可由初始状态到达。

- **活结点管理**: 使用优先队列 (堆结构) 组织活结点, 每个结点按“成本估计值”排序, 总是扩展当前成本最小的结点
- **成本计算**: 对于已部分解 $x[1..k]$, 计算已发生成本 (从根到当前结点的实际代价) 和预估成本 (启发式函数估算完成解所需的最小代价), 总成本 = 已发生成本 + 预估成本 (结点的成本估计函数: $\hat{c}(X) = f(h(X)) + \hat{g}(X)$). 如果 X 是一个答案结点或是一个叶子结点, 则有 $c(X) = \hat{c}(X)$
 - $\hat{g}(X)$ 是由 X 到达一个答案结点所需做的附加工作的估计
 - $h(X)$ 是由根结点到结点 X 的成本
 - $f(\cdot)$ 是一个非降函数。 f 函数强使算法优先选择更靠近答案结点但又离根较近的结点。它也能调整 $h(X)$ 和 $\hat{g}(X)$ 在 $\hat{c}(X)$ 中的影响比例。

- **搜索过程**:

1. 初始化优先队列 (加入根结点)

2. 循环直到队列为空:

- 取出队首 (成本最小的 E-结点)
- 生成所有子结点
- 对每个子结点计算成本估计值, 检查剪枝条件 (若成本 > 当前最优解则丢弃), 存活结点入队

– 若到达叶结点则更新最优解

LC-检索的两种特殊情况:

1. 当 $f(h(x)) = \text{结点 } X \text{ 的级数}$, 且 $\hat{g}(X) = 0$ 时 (忽略未来成本, 只考虑历史成本, 因而先考虑浅层结点): 若结点的 $\hat{c}$ 值相同, 选取最先生成的活结点. 此时 LC-检索等价于 BFS 检索 (队列实现的宽度优先搜索)
2. 当 $f(h(x)) = 0$, 且满足 $\forall Y \in \text{children}(X), \hat{g}(X) \geq \hat{g}(Y)$ 时 (忽略历史成本, 只考虑未来成本, 因而先考虑深层结点): 若结点的 $\hat{c}$ 值相同, 选取最晚生成的活结点. 此时 LC-检索等价于 D-检索 (堆栈实现活结点表)

成本函数 $c(\cdot)$: 从根出发到最近目标结点路径上的每个结点赋予这条路径的长度作为它们的成本。

估计成本函数 $\hat{c}(\cdot)$: $\hat{c}(X) = h(X) + \hat{g}(X)$

- $h(X)$ 是由根到结点 X 的路径的长度
- $\hat{g}(X)$ 是对以 X 为根的子树中由 X 到目标状态的一条路径长度的估计。它至少应是能把状态 X 转换成目标状态所需的最小移动数
- $\hat{g}(X) =$ 不在其目标位置的非空白牌数目
- 可以看出 $\hat{c}(X)$ 是 $c(X)$ 的下界

状态空间树 T ，结点的代价函数 $\hat{c}$

可行解函数 $cost$ ， $\varepsilon > 0$ ，下界函数 lb

Input:

假设： T 至少包含一个可行解结点，且满足 $\forall X, lb(X) \leq c(X) \leq \hat{c}(X)$

ε 足够小，使得 $\forall X, Y, lb(X) < lb(Y) \Rightarrow lb(X) - \varepsilon < lb(X) < lb(Y)$

Output: 函数值最大的解

```
1  $E \leftarrow T$ ;  
2  $PARENT(E) \leftarrow 0$ ;  
3 if  $T$  是可行解结点 then  
4    $L \leftarrow \max(cost(T), lb(T) - \varepsilon)$ ;  
5    $ans \leftarrow T$ ;  
6 else  
7    $L \leftarrow lb(T) - \varepsilon$ ;  
8    $ans \leftarrow 0$ ;  
9 end  
10 初始化活结点表为空;  
11 while true do  
12   for  $E$  的每个儿子  $X$  do  
13     if  $c(X) > L$  then  
14        $ADD(X)$ ;  
15        $PARENT(X) \leftarrow E$ ;  
16       if  $X$  是可行解结点 且  $cost(X) > L$  then  
17          $L \leftarrow \max(cost(X), lb(X) - \varepsilon)$ ;  
18          $ans \leftarrow X$ ;  
19       end  
20       else if  $lb(X) - \varepsilon > L$  then  
21          $L \leftarrow lb(X) - \varepsilon$ ;  
22       end  
23     end  
24   end  
25   if 活结点表为空 或 下一个  $E$  结点满足  $\hat{c}(E) \leq L$  then  
26     输出从  $ans$  到  $T$  的路径;  
27     输出最大值  $L$ ;  
28     return;  
29   end  
30    $LARGEST(E)$  ; // 选择  $\hat{c}$  值最大的活结点作为  $E$   
31 end
```

Algorithm 1: UCBB（基于代价的搜索算法）

ε 确保 $lb(X) < lb(Y) \Rightarrow lb(X) - \varepsilon < lb(Y)$ ，消除了边界值相等时的判断歧义。

1.2 分支限界法的应用

分支限界法的核心在于 **cost 函数设计**，通过评估部分解的潜在最优性实现高效剪枝。其通用模板为：

1. 初始化优先队列（按 cost 排序）
2. 扩展当前最优部分解的子节点
3. 计算子节点的 cost 值并剪枝

1.2.1 Cost 函数设计方法

表 1: 经典 Cost 函数对比：实际计算与理论性质

方法	Cost 函数形式 C_{method}	紧致性 ρ
MST 下界	$C = L_{current} + W_{MST}$	中等
线性松弛	$C = V_{selected} + \sum \left(v_i \cdot \min \left(1, \frac{W_{remain}}{w_i} \right) \right)$	高
贪心启发式	$C = C_{current} + \hat{C}_{remain}$	低-中
曼哈顿距离	$C = g(n) + h(n)$	低

对于线性松弛方法中的 Cost 函数：

$$C = V_{selected} + \sum \left(v_i \times \min \left(1, \frac{W_{remain}}{w_i} \right) \right)$$

其中：

- $V_{selected}$ = 当前已选物品的总价值
- W_{remain} = 背包/容器的剩余可用容量
- v_i = 第 i 个物品的价值
- w_i = 第 i 个物品的重量/体积

公式含义：

1. $\min \left(1, \frac{W_{remain}}{w_i} \right)$ 计算每个物品能放入剩余容量的最大比例
 - 当 $\frac{W_{remain}}{w_i} \geq 1$ 时，可取完整物品（比例 = 1）
 - 当 $\frac{W_{remain}}{w_i} < 1$ 时，只能取部分物品（比例 < 1）
2. $v_i \times \min(\dots)$ 表示该物品的实际贡献价值
3. 求和项 $\sum$ 计算所有未选物品的潜在最大价值

典型应用场景：

- 背包问题（Knapsack Problem）的线性规划松弛
- 资源分配问题中计算理论上界
- 分支定界法中的界限估计

1.2.2 关键实现技术

```

1 Function BranchAndBound(problem)
2    $Q \leftarrow \text{InitializePriorityQueue}();$ 
3   Insert( $Q$ , root_node);
4   while  $Q \neq \emptyset$  do
5      $N \leftarrow \text{ExtractMin}(Q);$ 
6     if IsCompleteSolution( $N$ ) then
7       | UpdateGlobalBest( $N$ );
8     else
9       foreach child node  $N_i$  of  $N$  do
10        |  $\text{cost}(N_i) \leftarrow \text{CalculateCost}(N_i);$  // 核心计算步骤
11        | if  $\text{cost}(N_i)$  better than current bound then
12          | Insert( $Q$ ,  $N_i$ );
13        | end
14      end
15    end
16  end
17  return global_best;

```

Algorithm 2: 分支限界法通用模板

采用 MST 下界作为 cost 函数：

$$\text{cost}(\pi) = \sum_{i=1}^k d_{\pi(i)} + \text{MST}(\text{未访问城市})$$

其中 π 为已访问路径， $d_{\pi(i)}$ 为路径长度。

使用线性松弛上界：

$$\text{cost}(S) = \sum_{i \in S} v_i + \sum_{j \notin S} v_j \cdot \min \left(1, \frac{W - \sum_{i \in S} w_i}{w_j} \right)$$

1.2.3 TSP 问题中的 MST 下界

$$C_{\text{MST}} = L_{\text{current}} + W_{\text{MST}}$$

- 变量说明：

- L_{current} ：当前已构建路径的总长度
- W_{MST} ：剩余未访问节点最小生成树 (MST) 的边权和

- 计算原理：

1. 通过 Prim 或 Kruskal 算法计算剩余节点的 MST
2. MST 权值和 W_{MST} 是连接剩余节点的最小代价
3. 最终路径长度 $\geq L_{\text{current}} + W_{\text{MST}}$

- 示例:

```

1 def tsp_lower_bound(current_path, remaining_nodes):
2     mst = compute_mst(remaining_nodes) # 计算剩余节点MST
3     return current_path.length + mst.total_weight

```

1.2.4 背包问题中的线性松弛

$$C_{LP} = V_{\text{selected}} + \sum_{i \in \text{剩余物品}} \left(v_i \times \min \left(1, \frac{W_{\text{remain}}}{w_i} \right) \right)$$

- 变量说明:

- V_{selected} : 已选物品总价值
- W_{remain} : 背包剩余容量
- v_i/w_i : 物品的价值-重量比

- 计算原理:

1. 对剩余物品按 v_i/w_i 降序排序
2. 尽可能装入高价值密度物品（可分割）
3. min 函数确保不超过物品本身价值

- 分支定界法中的应用:

$$\text{全局上界} = \max_{\text{所有节点}} C_{LP}$$

- 算法实现:

```

1 def knapsack_upper_bound(selected, remaining):
2     sorted_items = sorted(remaining, key=lambda x: x.v/x.w, reverse=True)
3     bound = selected.total_value
4     for item in sorted_items:
5         if W_remain <= 0: break
6         take = min(1, W_remain/item.w)
7         bound += take * item.v
8         W_remain -= take * item.w
9     return bound

```